14.2 - Syntax and Usage of Generics in Java

Generics in Java provide type safety and improved code readability by allowing you to specify the type of objects that a collection can hold. This eliminates the need for type casting and helps catch type-related errors at compile time. Let's explore the syntax and usage of generics, particularly with collections.

👉1. Syntax of Generics

The general syntax for using generics with collections is as follows:

ArrayList<Type> list = new ArrayList<>();

- The <Type> part is known as the **generic type parameter**, where you specify the type of objects the list will hold.
- This ensures **type safety** by allowing only objects of the specified type to be added to the collection. If you try to add an object of a different type, the compiler will show an error.

👉 Example Syntax:

ArrayList<String> list = new ArrayList<>();

In this example, the ArrayList is designed to hold String objects. The type parameter <String> specifies that only String objects are allowed.

Note: From Java 7 onwards, the type parameter on the right-hand side can be omitted and replaced with <>, known as the **diamond operator**. This infers the type from the left-hand side of the declaration:

ArrayList<String> list = new ArrayList<>();

👉 2. Example: Using Generics with Collections

Let's see an example where we use generics with ArrayList:

```
import java.util.ArrayList;

ArrayList<String> list = new ArrayList<>();
list.add("Rohan");
list.add("Rahul");
list.add("Remi");

// Accessing elements without the need for type casting
String name1 = list.get(0);
String name2 = list.get(1);
String name3 = list.get(2);
```

```
// Printing the names in uppercase
System.out.println(name1.toUpperCase());
System.out.println(name2.toUpperCase());
System.out.println(name3.toUpperCase());
```

Output:

```
ROHAN
RAHUL
REMI
```

In the above code:

- The ArrayList<String> allows only String objects to be added.
- If you try to add a different type, such as an Integer, the compiler will give an error.

👉 3. Generics with Custom Classes

Generics can also be used with user-defined classes, ensuring that collections hold only objects of a specific class type.

👉 Example: Using Generics with Custom Classes

Consider two custom classes, Student and Employee:

```
class Student {
    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters and other methods...
}

class Employee {
    private int id;
    private String name;
```

```
    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters and other methods...
}
```

👉 Using Generics with ArrayList

```
import java.util.ArrayList;

public class Demo {
    public static void main(String[] args) {
        Student st1 = new Student(1, "Rohan");
        Student st2 = new Student(2, "Aarav");

        Employee em1 = new Employee(1, "Harsh");
        Employee em2 = new Employee(2, "Sushil");

        ArrayList<Student> studentList = new ArrayList<>();
        studentList.add(st1);
        studentList.add(st2);

        // The following line will cause a compile-time error:
        // studentList.add(em1); // Error: incompatible types:
Employee cannot be converted to Student

        // Example of a non-generic list
        ArrayList<Object> list2 = new ArrayList<>();
        list2.add(10);
        list2.add("Shami");
        list2.add(9822244441111L);

        // Using a generic list for integers
        ArrayList<Integer> list3 = new ArrayList<>();
        list3.add(10);

        // Invalid example (uncommenting the line below will
cause a compile-time error):
        // ArrayList<int> list4 = new ArrayList<>(); // Error:
primitive types are not allowed
    }
}
```

In this example:

- The studentList is defined to hold only Student objects. Trying to add an Employee object results in a compile-time error.
- A non-generic ArrayList (list2) can hold objects of any type, leading to potential type safety issues.
- Generic collections only accept **reference types**, such as Integer, String, or user-defined classes. **Primitive types** (like int, char) are not allowed.

👉 4. Important Notes on Generics

- **Generics Work Only with Reference Types:** When using generics, the type parameter must be a reference type. For example, you cannot use int or char directly; you need to use their wrapper classes (Integer, Character, etc.).
- **Type Inference:** With the **diamond operator (<>)**, Java can infer the type from the left-hand side, making the code more concise.